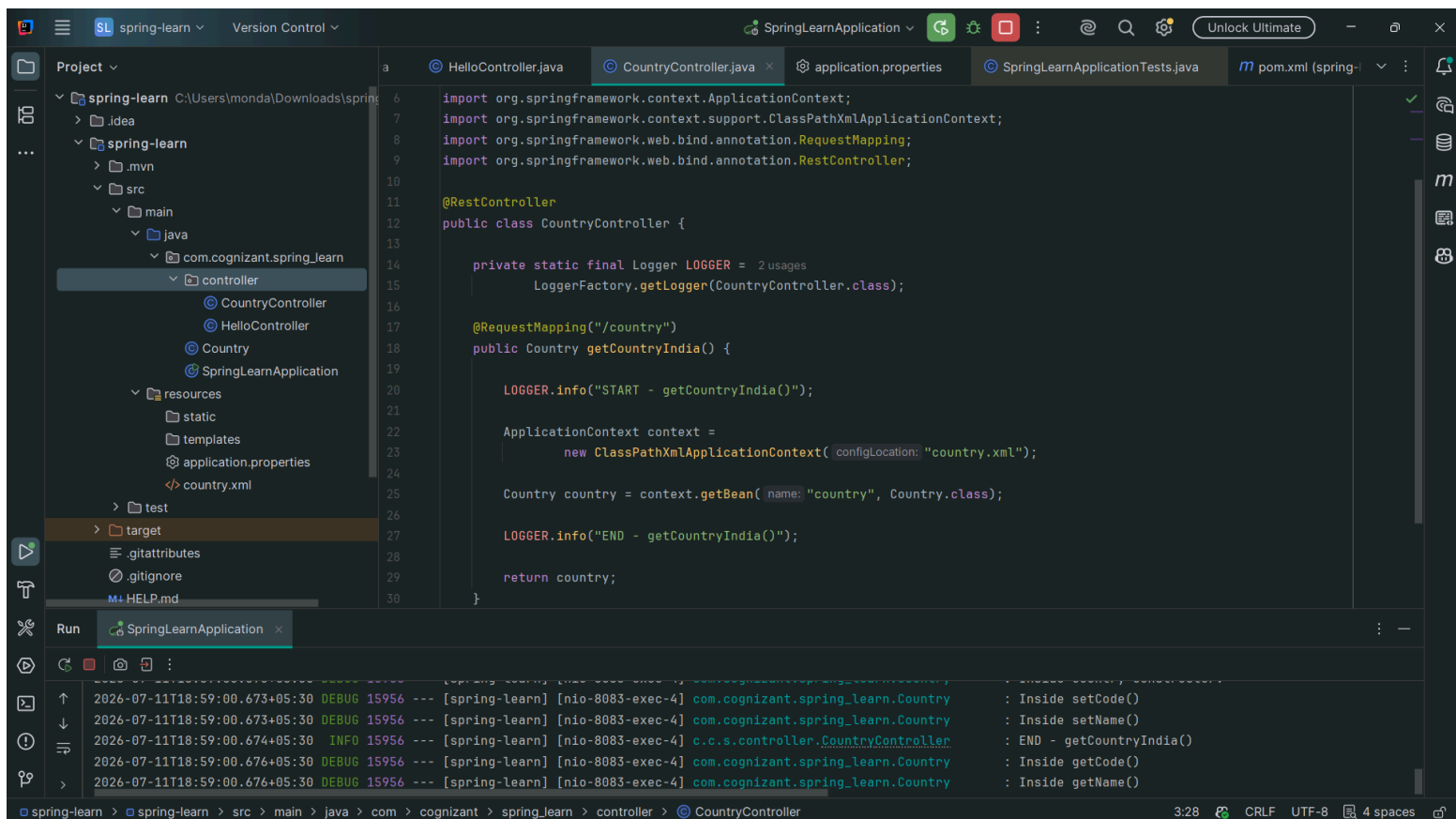


REST - Country Web Service

Code



```
import org.springframework.context.ApplicationContext;
import org.springframework.context.support.ClassPathXmlApplicationContext;
import org.springframework.web.bind.annotation.RequestMapping;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class CountryController {

    private static final Logger LOGGER = 2 usages
        LoggerFactory.getLogger(CountryController.class);

    @RequestMapping("/country")
    public Country getCountryIndia() {

        LOGGER.info("START - getCountryIndia()");

        ApplicationContext context =
            new ClassPathXmlApplicationContext( configLocation: "country.xml");

        Country country = context.getBean( name: "country", Country.class);

        LOGGER.info("END - getCountryIndia()");

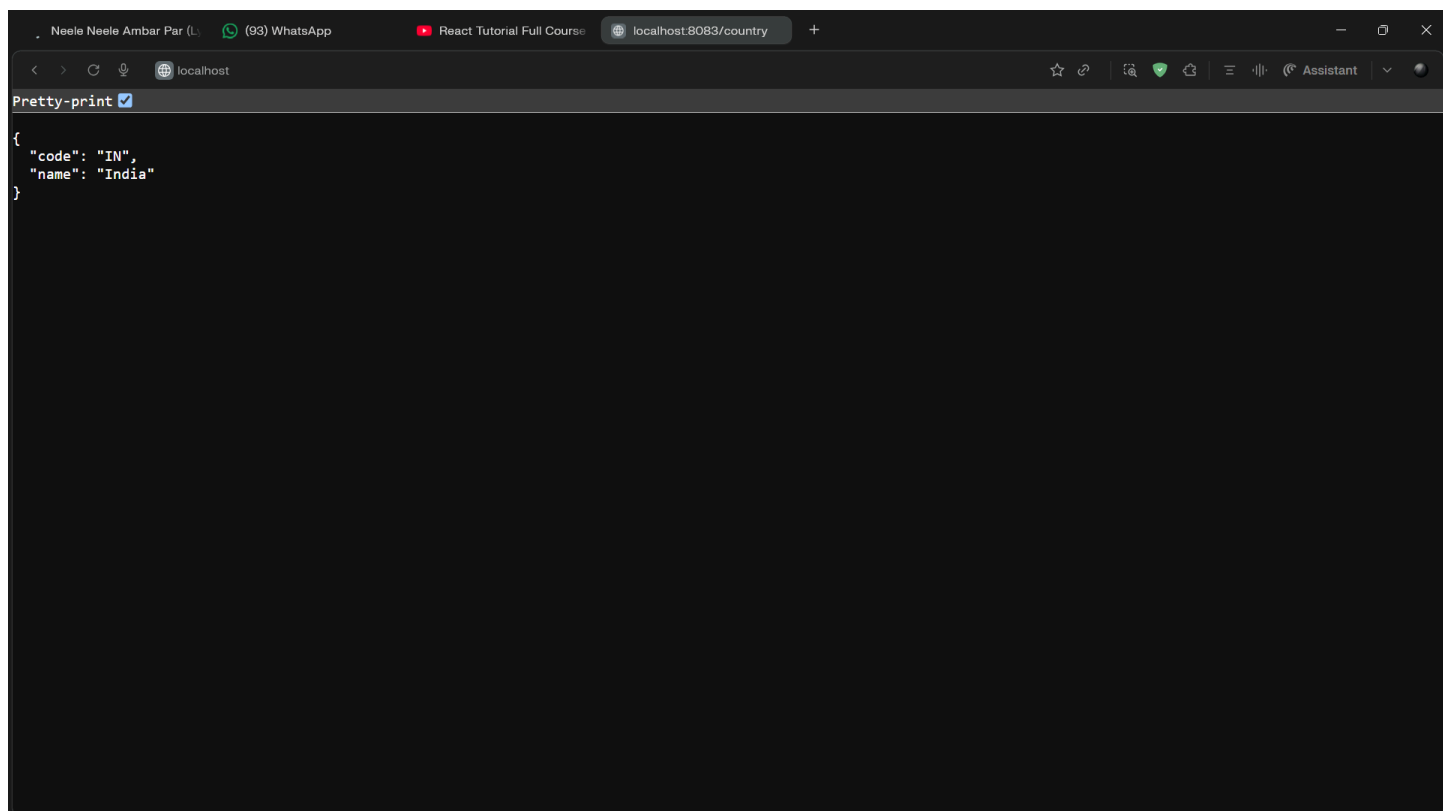
        return country;
    }
}
```

Run: SpringLearnApplication

```
2026-07-11T18:59:00.673+05:30 DEBUG 15956 --- [spring-learn] [nio-8083-exec-4] com.cognizant.spring_learn.Country : Inside setCode()
2026-07-11T18:59:00.673+05:30 DEBUG 15956 --- [spring-learn] [nio-8083-exec-4] com.cognizant.spring_learn.Country : Inside setName()
2026-07-11T18:59:00.674+05:30 INFO 15956 --- [spring-learn] [nio-8083-exec-4] c.c.s.controller.CountryController : END - getCountryIndia()
2026-07-11T18:59:00.676+05:30 DEBUG 15956 --- [spring-learn] [nio-8083-exec-4] com.cognizant.spring_learn.Country : Inside getCode()
2026-07-11T18:59:00.676+05:30 DEBUG 15956 --- [spring-learn] [nio-8083-exec-4] com.cognizant.spring_learn.Country : Inside getName()
```

Output

Chrome localhost:8083/country

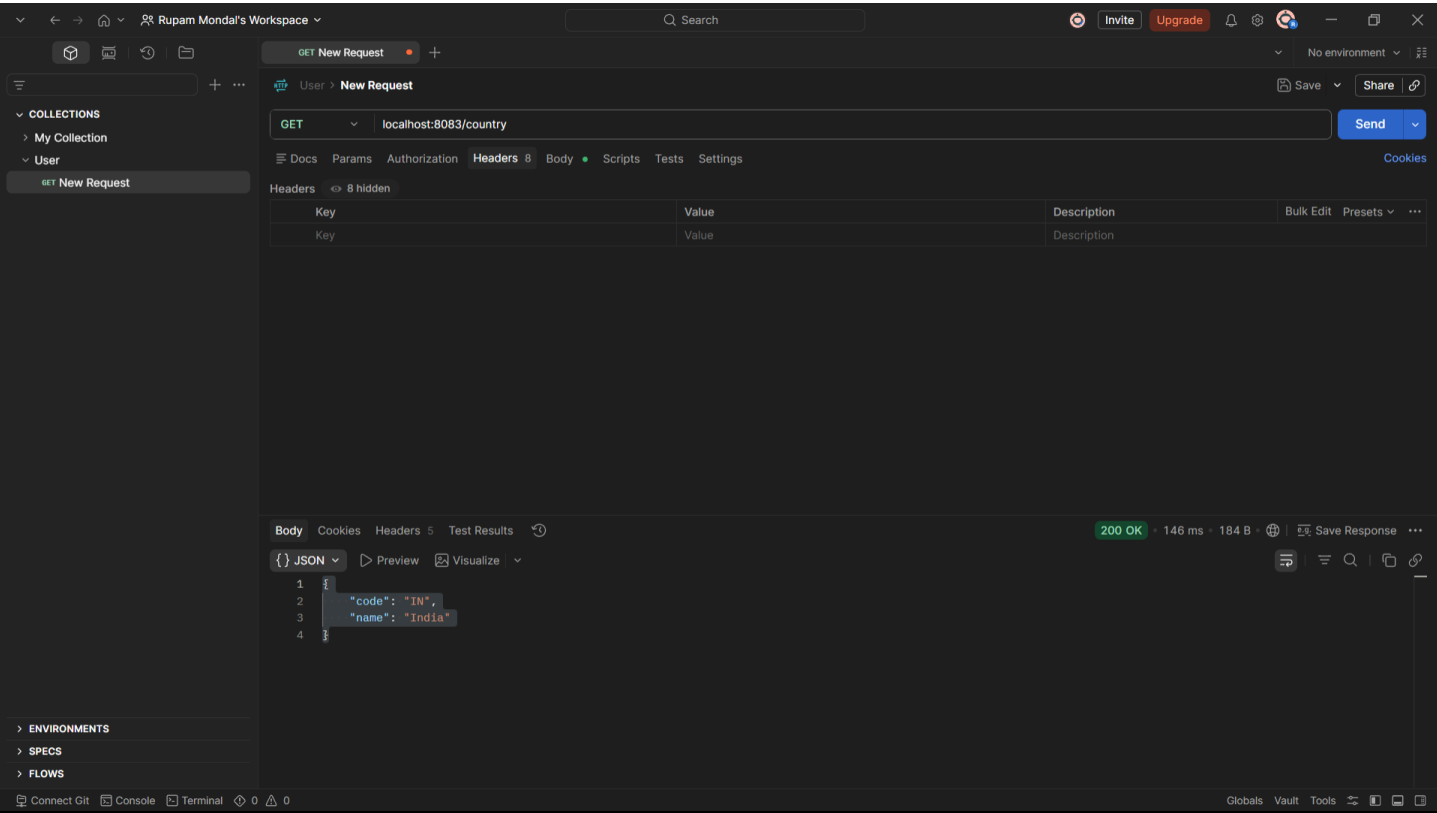


```
{
  "code": "IN",
  "name": "India"
}
```

REST - Country Web Service

Postman

Get request



SME

SME Explanation

1. What happens in the controller method?

When the browser or Postman sends a request to:

GET /country

Spring performs the following steps:

1. Receives the HTTP request.
2. DispatcherServlet finds the controller mapped to /country.
3. Calls the getCountryIndia() method.
4. Creates an ApplicationContext using country.xml.
5. Retrieves the country bean.
6. Returns the Country object.
7. Spring converts the object into JSON and sends it in the HTTP response.

Flow:

Browser/Postman

|

GET /country

|

DispatcherServlet

|

CountryController

|

getCountryIndia()

|

ApplicationContext

|
country.xml

|
Country Bean

|
JSON Response

2. How is the bean converted into JSON?

The spring-boot-starter-web dependency includes the **Jackson** library.

When the controller returns:

```
return country;
```

Spring uses Jackson to:

1. Inspect the object's getters (getCode() and getName()).
2. Create a JSON object.
3. Set the response header:

Content-Type: application/json

The JSON sent to the client is:

```
{  
  
  "code": "IN",  
  
  "name": "India"  
}
```

No extra code is needed for JSON conversion.

3. HTTP Headers (Browser)

Open **Developer Tools (F12)** → **Network** → refresh the page → click the country request.

Typical response headers:

HTTP/1.1 200 OK

Content-Type: application/json

Content-Length: 35

Date: Thu, 09 Jul 2026

4. HTTP Headers (Postman)

After clicking **Send**, open the **Headers** tab.

Typical response headers:

Header	Value
Content-Type	application/json
Content-Length	35
Date	Current Date
Connection	keep-alive

Status:

200 OK